РЕШЕН ИЗПИТ 4 — С ПЪЛНИ ОБЯСНЕНИЯ

Базови Програмни Езици — C (plain C)

Изпит 4: Клиника — Домашни любимци | 100 точки

Теми: createPatient() връща стойност | търсене по float цена | четене от бинарен с >= условие

Специфика: createPatient() ВРЪЩА Patient по стойност — не указател!

Задача 3 носи 40 точки. Разделителят е ',' (запетая), не ';'!

СЕКЦИЯ 0 — ПРОЧИТ И АНАЛИЗ

КОД / ИЗРАЗ	КАКВО ОЗНАЧАВА
`Собственик (до 30 символа)`	char ownerName[31]; — 30 + 1 = 31
`Чип "A12345B" (фиксиран)`	char chip[8]; — A12345B = 7 символа + 1 = 8
`Цена за преглед (0.00)`	float price; — %.2f при printf
`Функцията не приема параметри`	Patient createPatient() — без аргументи!
`Функцията ВРЪЩА създадения елемент`	return p; — по стойност, не указател
`разделени с ',' (запетая)`	fprintf(f, "%s,%.2f\n", ...) — не ';'!
`равна или по-голяма от параметъра`	price >= minPrice — включително равна!

СЕКЦИЯ 1 — ПЪЛЕН КОД (gcc -Wall -std=c99)

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char  ownerName[31]; /* 30 + 1 */
    char  chip[8];       /* A12345B = 7 + 1 */
    float price;
} Patient;

/* — Задача 1: createPatient() — без параметри, връща стойност — */
Patient createPatient() {
    Patient p;
    printf("Собственик: "); scanf("%30s", p.ownerName);
    printf("Чип:        "); scanf("%7s",  p.chip);
    printf("Цена:       "); scanf("%f",  &p.price);
    FILE *f = fopen("animalsText.txt", "a");
    if (!f) { printf("Error\n"); return p; }
    fprintf(f, "%s,%.2f\n", p.ownerName, p.price);
    fclose(f);
    return p;
}

/* — Задача 2: findByPrice — */
void findByPrice(Patient *arr, int count, float price) {
    int found = 0;
    for (int i = 0; i < count; i++) {
        if (arr[i].price == price) {
            printf("%s - %s\n", arr[i].ownerName, arr[i].chip);
            found = 1;
        }
    }
    if (!found) printf("Не е намерен.\n");
}

/* — Задача 3: четене от бинарен файл — */
void printAbove(char *fn, float minPrice) {
    FILE *f = fopen(fn, "rb");
    if (!f) { printf("Error\n"); return; }
    Patient p;
    while (fread(&p, sizeof(Patient), 1, f) == 1) {
        if (p.price >= minPrice) {
            printf("Bin OwnerName: %s\n", p.ownerName);
            printf("Bin Chip: %s\n",      p.chip);
            printf("Bin Price: %.2f\n",   p.price);
        }
    }
    fclose(f);
}

int main() {
    Patient *arr = NULL;
    int count = 0;
    for (int i = 0; i < 3; i++) {
        count++;
        arr = realloc(arr, count * sizeof(Patient));
        if (!arr) { printf("Error\n"); return 1; }
        arr[count - 1] = createPatient();
```

```
    }
    findByPrice(arr, count, 45.50f);
    printAbove("animalsBin.bin", 40.0f);
    free(arr);
    return 0;
}
```

СЕКЦИЯ 2 — ОБЯСНЕНИЯ РЕД ПО РЕД

createPatient() — два различни подхода за Задача 1

```
/* Подход А (Изпит 5): функцията управлява масива */
Member* addMember(Member *arr, int *count) { /* realloc вътре */ }
arr = addMember(arr, &count);

/* Подход Б (Изпит 4): функцията САМО създава елемент */
Patient createPatient() { return p; }  /* без параметри */
arr = realloc(...); arr[count-1] = createPatient(); /* caller управлява */
```

Ред по ред: ключовите моменти

КОД	ОБЯСНЕНИЕ
`Patient p;`	Локална структура на стека. Ще се КОПИРА при return.
`return p;`	Копие на p се връща на caller-а. Локалната p изчезва от стека.
`arr[count-1] = createPatient();`	Присвояваме върнатото копие. В C = копира всички полета на struct.
`fprintf(f, "%s,%.2f\n", ...)`	Разделителят е ',' (запетая) — прочети ВНИМАТЕЛНО условието!
`p.price >= minPrice`	>= означава равна ИЛИ по-голяма. Задачата казва 'равна или по-голяма'.

Грешки характерни за Изпит 4

```
[NE]  Patient* createPatient()          // задачата иска Patient (стойност)!
[OK]  Patient createPatient()           // без звезда — return p; по стойност

[NE]  fopen("animalsBin.bin", "r")      // "r" е за текстов — данните се четат
грешно!
[OK]  fopen("animalsBin.bin", "rb")     // "rb" задължително за бинарни

[NE]  fprintf(f, "%s;%.2f\n", ...)     // разделителят е ",", не ";"!
[OK]  fprintf(f, "%s,%.2f\n", ...)     // запетая от условието

[NE]  if (p.price > minPrice)           // изключва РАВНАТА цена!
[OK]  if (p.price >= minPrice)          // условието казва 'равна или по-голяма'
```